EliteMobile — Project Structure

React Native / Expo · clean architecture module layout

The core idea: every business feature is a self-contained **module** under `src/modules/<name>/`, and every module follows the exact same five-folder shape (`core/entities`, `core/ports`, `core/usecases`, `infrastructure/adapters`, `ui`). Once you know this shape, you can navigate *any* module without re-learning its layout.

Top-level layout

```
EliteMobile/
├── app/                        # expo-router routes — thin wrappers, no business logic
│   ├── _layout.tsx             # root layout: mounts every global provider +
│   ├── index.tsx               # Splash screen route
│   ├── (tabs)/                  # the 4 bottom-tab screens
│   │   ├── _layout.tsx         # bottom tab bar (protected — see note below)
│   │   ├── home.tsx
│   │   ├── roster.tsx
│   │   ├── tickets.tsx
│   │   └── timesheet.tsx
│   └── *.tsx                   # pushed/modal routes: sign-in, device-registration,
│                                  # attestation, ticket-detail, travel, notes, profile, sync-queu
│
├── src/
│   ├── modules/                  # one folder per business feature — see below
│   ├── ui/
│   │   ├── components/
│   │   │   ├── atoms/            # smallest reusable pieces (buttons, glass surface, keypad…)
│   │   │   ├── molecules/        # small compositions of atoms (rows, cards, banners…)
│   │   │   └── organisms/        # bigger shared pieces (TopBar, ScreenBackground)
│   │   ├── theme/                # colors.ts, typography.ts, layout.ts, glass.ts
│   │   └── utils/                # small cross-cutting UI helpers (e.g. confirmSignOut)
│   └── types/                    # Result<T,E> and other app-wide TS types
│
├── __mocks__/                    # manual Jest mocks for native modules
├── docs/                         # generated reference PDFs, superpowers specs/plans
├── ios/ · android/                # native projects (generated — don't hand-edit)
├── app.config.ts                # Expo app config (name, bundle id, plugins)
├── package.json
└── tsconfig.json
```

The module shape

Every folder under `src/modules/` looks like this (shown for the `home` module — the shape is identical for every other module):

```
src/modules/home/
├── core/
│   ├── entities/
│   │   └── HomeSummary.entity.ts       # plain TS types — the domain's data shape
│   ├── ports/
│   │   └── HomeSummaryReader.port.ts   # interface the usecase depends on (not an implementatio
│   └── usecases/
│       └── GetHomeSummary.usecase.ts   # one class per action, returns Result<T,E>
├── infrastructure/
│   └── adapters/
│       └── InMemoryHomeSummary.adapter.ts # implements the port — mock data today, a real API/D
└── ui/
    ├── viewModels/
    │   └── useHome.viewModel.tsx       # a hook: wires usecases + local state, returns {state,
    └── screens/
        └── HomeScreen.tsx              # pure presentation — consumes the viewModel, renders JS
```

Why this shape

- **core/** never imports React or any native/Expo module — it's plain, testable TypeScript.
- **infrastructure/adapters** is the only place that knows *how* data is actually fetched (today: hardcoded mock arrays; later: a real backend) — swapping one out never touches `core/` or `ui/`.
- **ui/viewModels** is where screen-specific state, timers, and business rules live — screens themselves stay "dumb," which keeps them easy to redesign without touching logic.
- Dependencies are wired once, centrally, in `src/modules/app/dependencies/dependencies.dev.ts`, and every viewModel/screen reads them via `useDependencies()` — nothing imports a concrete adapter class directly.

Every module, at a glance

Module	What it owns
app	Cross-cutting infrastructure shared by every screen: dependency injection (`DependenciesProvider`), the persistent timer engine, the toast/notification system, and EN/ES localization (`LanguageProvider` + the `translations/` dictionary).
shared	The `KeyValueStore` port + its MMKV-backed (encrypted) and in-memory adapters — the one persistence primitive every other module builds on.

splash	Cold-start screen: shows load progress, decides whether to skip straight to Sign In if the device is already registered.
deviceRegistration	First-run device setup: generates a hardware-backed identity keypair, submits for office approval.
auth	Employee-code sign-in (keypad entry, session creation).
home	The Home tab: crew-status banner, day-time entries, the next job's live timer card, "notify office" panel.
roster	Crew roster: select/bulk-select workers to clock in or out, add a worker from the directory.
clock	Individual attestation flow — each worker confirms their own punch with their employee code.
tickets	Job tickets: today's ticket list, ticket detail (job timer, meal-break flow, crew), and the Travel Time screen.
timesheet	Daily timesheet: per-worker acknowledge/dispute flow before supervisor submission.
notes	Ticket notes & photos: free-text note, photo/video tiles, "extra work" flag.
sync	Sync Queue: pending/rejected punch records waiting to sync to the office.
profile	Crew leader's profile: employee code, device, language, recent notifications, sign out.

Naming conventions worth knowing

- `*.entity.ts` — a plain data shape (interface/type), no behavior.
- `*.port.ts` — an interface describing a capability a usecase needs (e.g. "something that can read a roster").
- `*.usecase.ts` — one class, one `execute()` method, returns `Result<T, E>` (never throws for expected failures).
- `InMemory*.adapter.ts` — today's mock implementation of a port; swappable later for a real API/DB adapter of the same port.
- `use*.viewModel.tsx` — a hook, not a component; returns `{ state, handlers }`.
- `*Screen.tsx` — the actual screen component, consumed by an `app/*.tsx` route file.
- `*.test.tsx` / `*.test.ts` — colocated right next to the file they test, not in a separate `__tests__/` tree.

Files that are off-limits

Two files are permanently protected in this project and must never be modified: `src/ui/components/atoms/GlassSurface.tsx` and `app/(tabs)/_layout.tsx` (the bottom tab bar). Both were finalized early in the project's design-fidelity work and are treated as locked.